ADEPA FOOD HUT

Software Requirements Specification

Detailed functional, data, security, integration and quality requirements for the ADEPA Food Hut WordPress platform.

DOCUMENT	SRS
VERSION	0.1 — Discussion Draft
DATE	5 November 2026
PLATFORM	WordPress with custom PHP functionality
HOSTING	Hostinger Pro (exact plan and environment to be verified)
STATUS	For stakeholder review and technical discussion

Core principle: One connected system. Three user experiences — public website, customer portal and administrator dashboard.

Prepared as a working baseline for discussion. Items requiring client confirmation or technical validation are explicitly identified rather than treated as final decisions.

1. Introduction

1.1 Purpose

This SRS defines the proposed software behaviour and technical constraints for ADEPA Food Hut. It is intended for the project owner, developers, testers and implementation partners. It complements the client-facing PRD.

1.2 Scope

The system comprises a WordPress-based public website, customer account and ordering experience, and a protected administrative experience. Custom PHP functionality may be implemented in a dedicated plugin. The system will support delivery orders and offline payments only at launch.

1.3 Definitions

Term	Definition
Order	A customer request for one or more menu items and delivery.
Pending / Awaiting Approval	An order received by the system but not yet accepted by staff.
Accepted	An administrator has approved fulfilment of the order.
Offline payment	Payment handled outside the website, such as cash or an approved bank transfer.
Availability	Whether a menu item can currently be ordered.
Business window	Configured period during which the system may accept new orders.
Status history	Timestamped record of order status changes and actor.

2. Overall system description

2.1 Product perspective

The solution will run on WordPress hosted by Hostinger, subject to verification of the exact hosting plan and environment. WordPress supplies core content, users, media and administration capabilities. ADEPA-specific business rules should reside in a custom plugin or a carefully selected commerce engine, not solely in the theme.

2.2 Proposed logical architecture

Layer	Responsibilities
Presentation	Responsive WordPress theme/templates and agreed page-builder editing experience.
Application / domain	Menu, basket, order validation, order lifecycle, delivery decisions, business hours, payment records and notifications.
WordPress integration	User accounts, roles/capabilities, admin menus, hooks, REST endpoints, media and settings.
Persistence	WordPress database plus dedicated tables where transactional integrity and query patterns justify them.
External services	Email delivery and an SMS provider, subject to selection, credentials and provider capabilities.

2.3 User classes

Role	Permissions / access
Visitor	View public pages and menu; register or sign in. Cannot submit an order.
Customer	Manage own profile, submit orders and view only their own orders and status history.
Order staff	View and process assigned/permitted orders; update approved operational statuses.
Administrator	Manage business settings, menu, orders, customers, content, users/roles and integrations as authorised.
Developer / technical operator	Maintain WordPress, plugins, theme, backups and deployments; access should be limited and controlled.

3. Functional requirements

Priority definitions: **Must** = required for launch; **Should** = expected where feasible; **Could** = optional or later enhancement.

3.1 Public website and navigation

ID	Requirement	Priority
PUB-01	The system shall provide separate public pages for Home, Menu, About, Delivery, Contact, Privacy and Terms.	Must
PUB-02	Primary navigation links shall open their respective page routes, not merely scroll to sections on the homepage.	Must
PUB-03	The site shall provide consistent header, footer, visual styles and responsive behaviour across pages.	Must
PUB-04	Public pages shall be editable through the agreed WordPress editing approach for supported content areas.	Must
PUB-05	The menu shall remain viewable when ordering is closed, while order submission is disabled by server-side rules.	Must

3.2 Menu management

ID	Requirement	Priority
MEN-01	Administrators shall create, edit, publish, unpublish and archive menu items.	Must
MEN-02	Each menu item shall support name, description, category, price, image, publication state and availability state.	Must
MEN-03	Administrators shall manage menu categories and item ordering.	Must
MEN-04	Administrators shall mark individual items available or sold out without deleting the item.	Must
MEN-05	The public menu shall display current published item data and availability.	Must
MEN-06	The server shall revalidate item publication, availability and price at order submission.	Must
MEN-07	Historical order line items shall preserve the item name and price charged/quoted at order time, even if the menu later changes.	Must

3.3 Customer accounts and authentication

ID	Requirement	Priority
ACC-01	A visitor shall be able to register using the required customer details and a password.	Must
ACC-02	A successfully registered customer shall be able to proceed without mandatory email verification.	Must
ACC-03	Customers shall be able to sign in and sign out securely.	Must
ACC-04	Customers shall be able to view and update permitted profile fields.	Must
ACC-05	Customers shall only access their own order records and associated status history.	Must
ACC-06	The system shall provide password reset/recovery functionality.	Must
ACC-07	Administrative accounts shall use separate roles and stricter permissions from customer accounts.	Must

3.4 Basket and order submission

ID	Requirement	Priority
ORD-01	Authenticated customers shall be able to add available menu items to a basket.	Must
ORD-02	Customers shall be able to change quantities and remove basket items before submission.	Must
ORD-03	The system shall calculate line subtotals and order totals using server-side monetary calculations.	Must
ORD-04	The customer shall provide delivery address and contact details required for fulfilment.	Must
ORD-05	The customer shall select an enabled offline payment method.	Must
ORD-06	The system shall not provide online payment processing at launch.	Must
ORD-07	The system shall accept new orders only within configured business/order windows; advance ordering shall not be supported.	Must
ORD-08	At submission, the server shall revalidate current price, item availability, operating window, customer identity and required order fields.	Must
ORD-09	The system shall persist the order before displaying a successful submission result.	Must
ORD-10	The system shall assign a unique, non-guessable customer-facing order reference or secure order identifier.	Must
ORD-11	Duplicate submissions shall be mitigated using an idempotency or equivalent duplicate-prevention mechanism.	Should

3.5 Delivery management

ID	Requirement	Priority
DEL-01	The launch service shall be delivery-only; customer pickup/collection shall not be offered.	Must
DEL-02	The website shall communicate Milton Keynes and Northampton as the intended delivery areas.	Must
DEL-03	A submitted address shall be subject to manual serviceability confirmation by authorised staff.	Must
DEL-04	Delivery charge shall be manually confirmed; the system shall not invent or imply an unapproved automatic charge.	Must
DEL-05	The system shall support recording the confirmed delivery charge and communicating the confirmed amount to the customer before fulfilment proceeds.	Must
DEL-06	The system shall record delivery instructions and relevant address fields.	Must
DEL-07	Staff shall be able to record dispatch and delivery completion.	Must

3.6 Order review and lifecycle

ID	Requirement	Priority
LIF-01	New orders shall enter a pending / awaiting-approval state.	Must
LIF-02	Authorised staff shall be able to accept or reject a pending order.	Must
LIF-03	The system shall support the statuses Received, Awaiting Approval, Accepted, Preparing, Dispatched and Delivered, plus Rejected and Cancelled.	Must
LIF-04	The system shall enforce permitted status transitions and reject invalid transitions.	Must
LIF-05	Each status change shall record timestamp and acting user; a note/reason should be supported where relevant.	Must
LIF-06	Customers shall be able to view the current status and a chronological status timeline for their own order.	Must
LIF-07	The tracking view shall update automatically or refresh promptly after status changes.	Should
LIF-08	Staff shall be able to view and search orders by reference, date, status and customer where authorised.	Must
LIF-09	Order records shall preserve the order-time item descriptions, quantities and prices.	Must

3.7 Offline payment records

ID	Requirement	Priority
PAY-01	The system shall support configurable offline payment methods.	Must
PAY-02	Initial candidate methods include cash on delivery and bank transfer; any other method requires business approval.	Must
PAY-03	The system shall store payment method separately from payment status.	Must
PAY-04	Payment statuses shall include Awaiting Payment and Paid; Partially Paid and Refunded may be enabled if required.	Must/Should
PAY-05	Authorised staff shall be able to record payment receipt, amount, date/time and relevant reference or note.	Must
PAY-06	The system shall not represent an order as paid solely because a payment method was selected.	Must

3.8 Admin dashboard

ID	Requirement	Priority
ADM-01	The system shall provide a protected ADEPA administration experience for daily operations.	Must
ADM-02	The dashboard shall include overview, Orders, Order Tracking, Menu, Availability, Business Hours, Customers, Delivery, Website Content, Notifications and Settings modules.	Must
ADM-03	Administrative actions shall be enforced by server-side capability checks, not merely hidden interface controls.	Must
ADM-04	The dashboard shall provide clear confirmation and feedback for important actions.	Must
ADM-05	The system shall record an audit trail for material order and settings changes where practical.	Should

3.9 Business hours and availability

ID	Requirement	Priority
BUS-01	Administrators shall configure operating days and time windows.	Must
BUS-02	Administrators shall configure a last-order cut-off for each applicable service window.	Must
BUS-03	Administrators shall be able to create temporary closures or exceptional opening windows.	Must
BUS-04	Order acceptance shall be checked on the server at submission time.	Must
BUS-05	The system shall use the business's UK local time and handle GMT/BST daylight-saving transitions correctly.	Must
BUS-06	The public site may display open/closed information only when based on configured and current schedule data.	Must

3.10 Notifications

ID	Requirement	Priority
NOT-01	The system should send email notifications for order receipt and relevant order-status changes.	Should
NOT-02	The project shall attempt integration with an SMS provider selected by the business.	Should
NOT-03	SMS/email delivery failure shall not undo or prevent persistence of an order or status change.	Must
NOT-04	Notification attempts and failures should be logged for troubleshooting without exposing secrets or unnecessary personal data.	Should
NOT-05	Notification templates should be editable or configurable within the agreed scope.	Could

4. Data requirements

4.1 Proposed logical entities

Entity	Key information
Customer / WordPress User	User ID, name, email, phone, role, account state and profile data.
Menu Category	ID, name, description, sort order and publication state.
Menu Item	ID, name, description, category, current price, image reference, publication and availability.
Order	Internal ID, public reference, customer ID, status, timestamps, delivery details, payment method/status and totals.
Order Item	Order ID, menu item reference where retained, item-name snapshot, unit-price snapshot, quantity and line total.
Order Status History	Order ID, prior/new status, timestamp, acting user and optional reason/note.
Delivery Details	Order ID, address fields, serviceability decision, confirmed charge and staff note.
Payment Record	Order ID, method, status, amount, recorded timestamp, staff actor and optional reference.
Business Schedule	Day/date, opening window, cut-off, exception/closure and timezone.
Notification Log	Event, channel, destination reference, attempt result, timestamp and provider reference where available.

4.2 Data integrity rules

- Order totals and order-line values must be stored using an appropriate decimal or integer-minor-unit strategy; do not use binary floating-point for financial calculations.
- An order must retain a snapshot of item name and price at submission so later menu edits do not rewrite historical orders.
- Order status history should be append-only for ordinary staff workflows; corrections should be auditable.
- Deleting a menu item must not delete or corrupt historical order details.
- Customer access to orders must be scoped to the authenticated customer ID.
- Database schema changes must be versioned and deployed through controlled plugin migrations.

5. Interface requirements

5.1 Public and customer interface

- Responsive layout with mobile-first ordering usability.
- Consistent navigation across public pages.
- Clear menu categories, item pricing and availability labels.
- Accessible forms with field-level validation and helpful error messages.
- A basket summary that makes item quantities and totals understandable.
- An order tracking timeline that clearly distinguishes completed, current and upcoming states.

5.2 Administrative interface

- Role-appropriate dashboard navigation.
- Order queue with clear status, customer, order reference and submission time.
- Order detail view with item snapshots, delivery information, payment method and status controls.
- Clear distinction between pending delivery-charge confirmation and confirmed charges.
- Confirmation for rejection, cancellation, closure and other consequential actions.
- No reliance on the standard WordPress dashboard as the only operational interface, subject to final admin UX decision.

6. External interface and integration requirements

6.1 WordPress integration

The implementation may use WordPress hooks, capabilities, REST API endpoints, nonces, media APIs, user APIs and database APIs. Any custom REST endpoint must authenticate and authorise requests and validate/normalise input.

6.2 SMS provider

Provider selection is not yet final. The implementation should isolate provider-specific code behind a notification interface. Credentials must be stored securely and never committed to source control. The provider's sender-ID rules, destination coverage, pricing, delivery reports and regulatory requirements must be checked before production use.

6.3 Email

Email delivery should use a reliable configured mail transport. The project must test deliverability and document DNS configuration such as SPF, DKIM and DMARC where applicable.

7. Security requirements

- Use HTTPS for all production pages and authenticated sessions.
- Use WordPress password hashing and supported authentication mechanisms; never store plaintext passwords.
- Enforce least privilege through roles and capabilities.
- Check authorisation on every sensitive server-side action and customer order read.
- Validate, sanitise and escape data according to WordPress security practices.
- Use nonces for state-changing browser requests where applicable; nonces do not replace authorisation.

- Protect against SQL injection through prepared database queries and safe WordPress APIs.
- Apply rate limiting or abuse controls to login, registration and order submission where feasible.
- Keep WordPress core, plugins, theme and PHP runtime supported and patched.
- Do not expose provider credentials, internal notes or another customer's personal information.
- Maintain backup and recovery procedures and test restoration before launch.

8. Non-functional requirements

Area	Requirement / acceptance expectation
Performance	Optimise image dimensions and formats, minimise unnecessary scripts, use caching where it does not expose private or stale order data, and test key pages on mobile.
Availability	Hosting and backup capability must be confirmed; define a practical recovery procedure with the client.
Accessibility	Aim for WCAG 2.2 AA practices for core customer journeys, including keyboard access, contrast, labels and status announcements.
Compatibility	Support current major desktop and mobile browsers; test common screen widths and devices.
Maintainability	Separate business logic from theme and page-builder presentation; use version control and documented deployment steps.
Scalability	Use indexed query patterns and avoid loading all orders or menu records on every request.
Privacy	Publish accurate privacy information, restrict data access and define retention/deletion handling with the business.
Observability	Log relevant errors and notification failures without logging passwords, secrets or unnecessary sensitive data.

9. Business rules and edge cases

- A customer may not submit an order while ordering is closed, even if the menu page or a stale browser tab remains open.
- A sold-out or unpublished item may not be included in a newly submitted order.
- The server must recalculate prices and totals at submission; browser-supplied totals are not authoritative.
- If price or availability changes while a basket is open, the customer must be shown the updated information and asked to review before submission.
- A pending order is not an accepted order. Customer messaging must make this distinction explicit.
- A delivery address in a target area is not automatically guaranteed to be serviceable.
- If the delivery charge is not yet confirmed, the customer must be told clearly that the business will confirm it before final acceptance/fulfilment.
- A notification provider outage must not erase a saved order.
- An order marked delivered must retain its earlier status history.
- Only authorised roles may accept/reject orders, change payment status or modify business settings.

10. Acceptance test outline

Test ID	Scenario	Expected result
AT-01	Visitor registers with valid details.	Account is created and customer can proceed without email verification.
AT-02	Unauthenticated visitor attempts order submission.	Submission is blocked and user is directed to sign in/register.
AT-03	Customer submits order during permitted window with available items.	Order is saved once as pending/awaiting approval and reference is shown.
AT-04	Customer submits while ordering is closed.	Server rejects submission with a clear message; no order is created.
AT-05	Customer basket contains a dish that becomes sold out.	Submission is blocked or item is removed after clear customer review.
AT-06	Price changes after item is added to basket.	Server detects mismatch and customer reviews the updated total before submission.
AT-07	Administrator accepts a pending order.	Order status and history update; customer tracking shows accepted.
AT-08	Administrator attempts invalid status transition.	Action is rejected and existing order state remains intact.
AT-09	Customer attempts to view another customer's order.	Access is denied.
AT-10	Administrator confirms delivery charge.	Confirmed charge is saved and shown accurately in the order record/customer view as designed.
AT-11	Administrator records payment received.	Payment status changes to paid with actor and timestamp.
AT-12	SMS provider is unavailable.	Order/status change remains saved; failure is logged and retried or handled.
AT-13	Menu item is edited after an old order was completed.	Historical order retains original item and price snapshot.
AT-14	Business schedule crosses UK daylight-saving transition.	Order acceptance uses correct UK local time and configured cutoff.
AT-15	Administrator changes menu availability.	Public menu reflects the saved state and unavailable item cannot be ordered.

11. Deployment, operations and maintenance

Development should occur locally or in a controlled staging environment. The live website should not be the primary development environment. The team should maintain source control, a deployment checklist, a backup before production releases and a rollback procedure.

- Confirm Hostinger plan, PHP version, database version, SSH/SFTP, WP-CLI, staging and backup features.
- Keep secrets and production credentials outside source control.
- Use a staging site for integration and acceptance testing.
- Run database migrations in a controlled, versioned manner.

- Verify backups and perform a restoration test.
- After deployment, test registration, login, menu, order submission, admin processing, tracking, notifications and mobile pages.

12. Technical decisions requiring resolution

Decision	Option(s)	Current recommendation / next action
Page editing	Gutenberg / WordPress Site Editor; Elementor; other.	Prefer Gutenberg for a lean native WordPress foundation unless the client specifically prefers Elementor.
Commerce engine	WooCommerce customised; custom ADEPA PHP plugin.	Run a short fit-gap assessment. Do not finalise database design until this is decided.
Customer account UI	Custom ADEPA pages; standard WordPress account screens.	Custom ADEPA-designed pages, using WordPress authentication underneath.
Admin UI	Custom ADEPA operations dashboard; standard WP admin with added menus.	Custom operational dashboard, with WordPress admin retained for technical maintenance.
Offline methods	Cash, bank transfer, physical card terminal, other.	Confirm actual accepted methods and payment confirmation rules.
Delivery charge flow	Pending confirmation before acceptance; staff-confirmed charge before customer submits; other.	Agree exact customer messaging and whether order acceptance waits for charge agreement.
SMS provider	Provider not selected.	Compare UK coverage, sender ID, cost, delivery reports and API reliability.

13. Traceability to PRD

The SRS requirements implement the PRD's agreed launch decisions: WordPress platform, separate public pages, required customer accounts, one standard menu, no advance orders, delivery-only service, manual order approval, offline payments, manual delivery confirmation and interactive status tracking. Where the PRD leaves an issue open, this SRS records it as a decision requiring confirmation rather than silently choosing a solution.

Document status

This is SRS version 0.1 for discussion. It is sufficiently detailed to guide discovery and technical evaluation, but the final database schema, plugin boundaries, WooCommerce decision, page-builder choice, delivery-charge workflow and provider integrations should be approved after the existing website and Hostinger environment are audited.